esprit
Se former autrement



Injection de dépendances dans .NET Core

Tarek Ayari

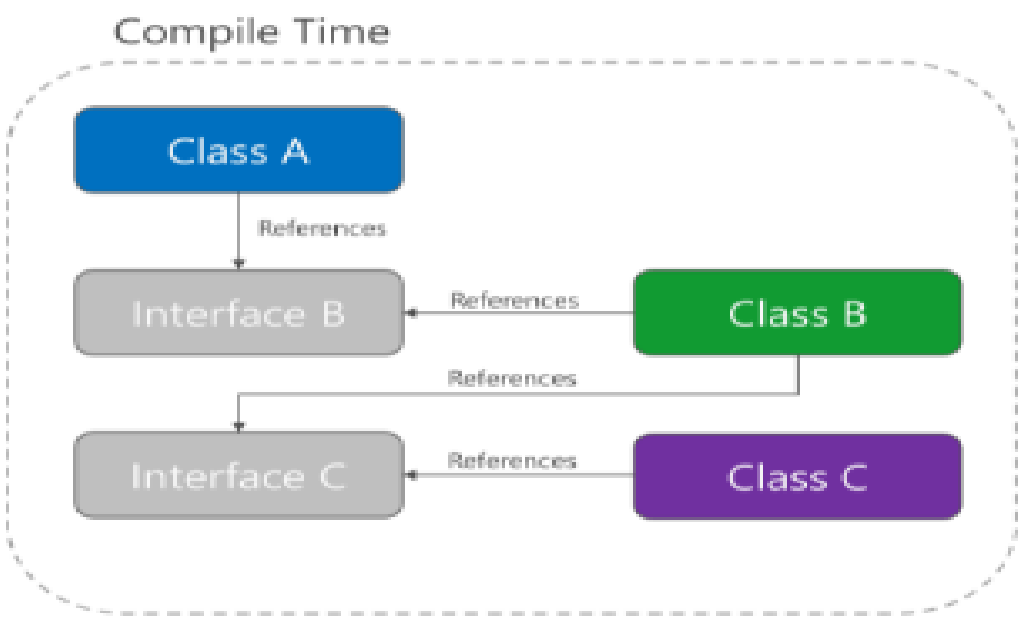
Présentation de l’injection des dépendances

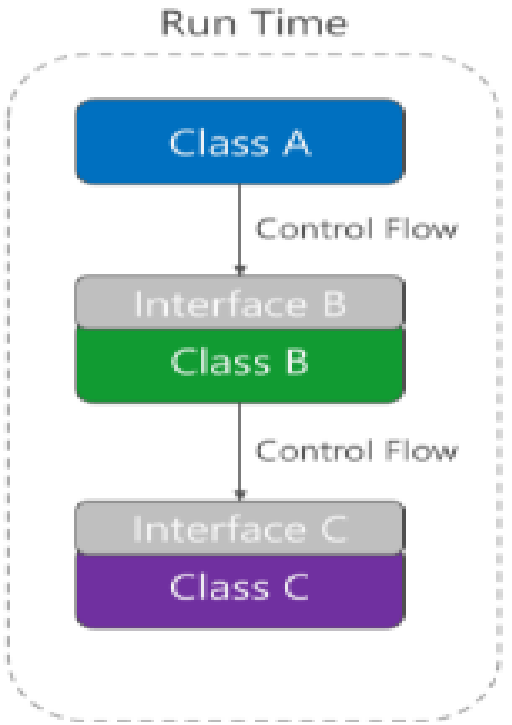
SOLID-DIP : DEPENDENCY INVERSION PRINCIPLE

- Il faut dépendre des abstractions, pas des implémentations
- Une caractéristique clé du DIP est la programmation d'abstraction pour que les consommateurs puissent compter sur ces abstractions plutôt que leurs implémentations.
- L'idée est que chaque point de contact entre deux modules soit matérialisé par une abstraction
- Une abstraction se matérialise par une interface ou une classe de base qui est aussi l'abstraction de classe plus élevé
- le DIP réduit couplage entre différents morceaux de code.

INJECTION DE DÉPENDANCES DANS .NET CORE

- .NET Core **prend en charge** le modèle de conception de logiciel d'injection de dépendance (DI), qui est une technique permettant de réaliser **l'inversion de dépendance** entre les classes et leurs dépendances.





CONTENEUR D'INJECTION DE DÉPENDANCES

- Un conteneur DI est un framework pour créer des dépendances et les injecter automatiquement si nécessaire.
- Il crée automatiquement des objets en fonction de la demande et les injecte si nécessaire.
- DI Container nous aide à gérer les dépendances au sein de l'application de manière simple et facile.
- Le conteneur DI crée un objet de la classe définie et injecte également toutes les dépendances requises sous la forme d'un objet qui est déclenchée au moment de l'exécution et se supprime au moment approprié.

ENREGISTREMENT DES SERVICES

- Plusieurs méthodes d'extension peuvent être utilisées pour enregistrer des services dans le conteneur: **AddSingleton, AddTransient** et **AddScoped.**

1/ Singleton (One Instance, Almost Forever)

- **AddSingleton():** Une nouvelle instance est créée lors de la première demande . Les objets singleton sont les mêmes pour chaque objet et chaque demande.

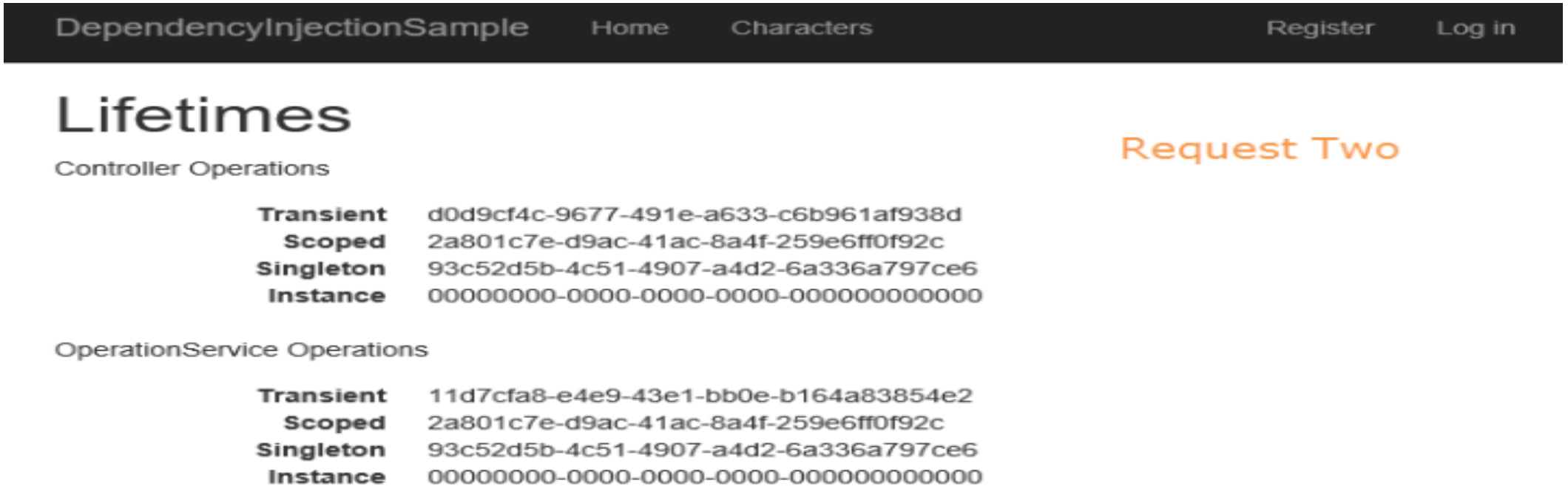
2/ Transient (New Instance Every Time)

- **AddTransient():** une nouvelle instance est fournie à chaque demande.

ENREGISTREMENT DES SERVICES

3/ Scoped (Once Per Application Request)

- **AddScoped():** Les objets scopes sont les mêmes dans une même requête, mais différents d'une requête à l'autre.



INJECTION DE DÉPENDANCES DANS ASP.NET CORE

INJECTION DE DÉPENDANCES DANS ASP.NET CORE

- Premièrement, les services qui devraient être injectés sont collectés a travers la classe ***WebApplicationBuilder*** qui conserve une liste des services.

```
var builder = WebApplication.CreateBuilder();

// Injection de dépendance
builder.Services.AddDbContext<DbContext, AMContext>();
builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
builder.Services.AddScoped<IServiceFlight, ServiceFlight>();
builder.Services.AddScoped<IServicePlane, ServicePlane>();
builder.Services.AddSingleton<Type>(t => typeof(GenericRepository<>));
```

- Une fois les services sont injectes , nous lançons un build pour récupérer une instance de la ***WebApplication.***

```
var app = builder.Build();
```

EXEMPLE DE SERVICE PREDEFINI DANS .NET CORE

```
// Adds  Controllers With Views
builder.Services.AddControllersWithViews();

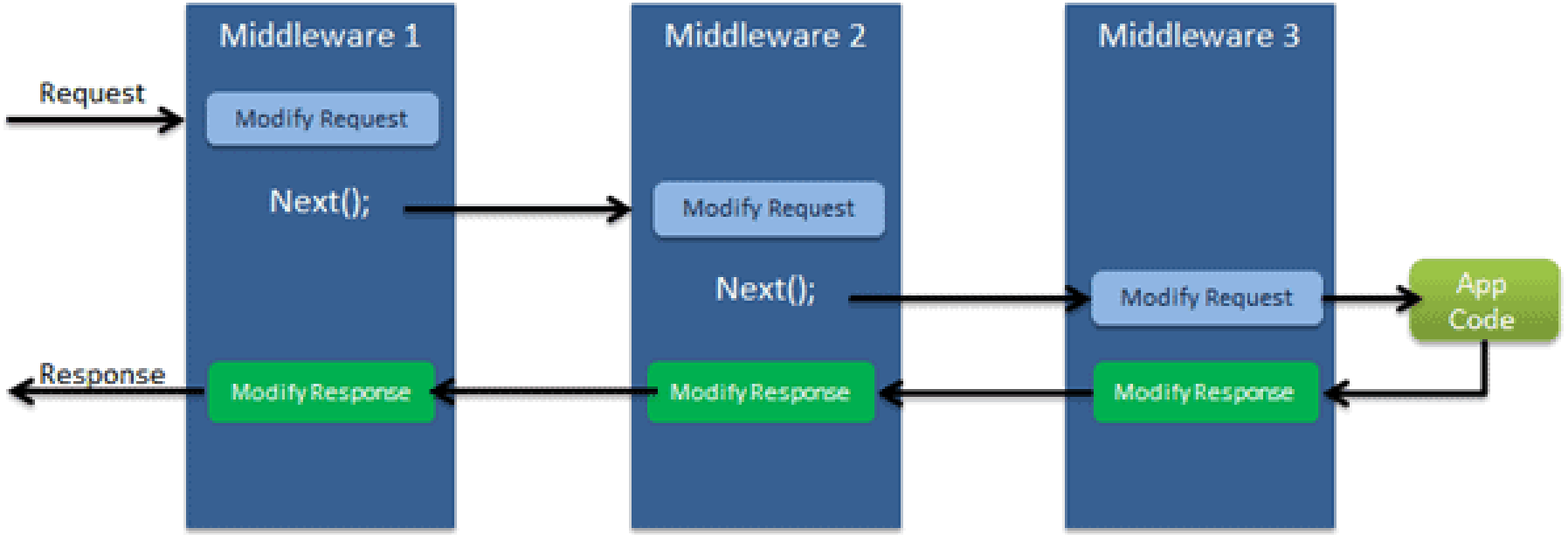
//Adds basic MVC functionality
builder.Services.AddMvc();

//Adds default logging support
builder.Services.AddLogging();

//Adds support for SignalR
builder.Services.AddSignalR();
```

PIPELINE D'APPEL ASP.NET

L'application asp.net Core utilise un **pipeline** de requêtes en ajoutant des composants pour arrive a l'execution finale



Middleware ASP.NET Core

EXEMPLE APPEL PIPELINE (ASP.NET)

```
var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
}

//Working with Static Files
app.UseStaticFiles();

// Matches request to an endpoint.
app.UseRouting();

// Use the default authorization.
app.UseAuthorization();

// create default route
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```